Relevant Source Code Analysis for Thesis Appendices

Selection Methodology

This analysis evaluates the STALA repository according to thesis relevance rather than repository completeness. Files were prioritized when they directly demonstrate the research contribution: optical music recognition workflow, document validation, staff segmentation, symbol-to-music interpretation, fretboard mapping, tablature generation, and session/export integration.

The review focused on project-owned source files in `lib/` and `android/app/src/main/kotlin/com/example/stala_app/`. Vendored OpenCV files, generated files, launcher assets, theme-only files, splash screens, simple wrappers, and dependency/build configuration were excluded because they do not materially demonstrate the thesis contribution.

Selection was based on:

- Algorithmic originality and complexity.
- Centrality to the OMR-to-tablature workflow.
- Role in coordinating multiple pipeline stages.
- Architectural significance across Flutter and native Android layers.
- Ability to demonstrate the system's research objectives without excessive code volume.
- Avoidance of duplicated or purely presentational code.

Recommended Files for Full Appendix Inclusion

1. `android/app/src/main/kotlin/com/example/stala_app/DocumentProcessor.kt`

- **Purpose:** Performs document detection, music-sheet-aware validation, second-pass crop validation, and document cropping.
- **Core Functionality:** Detects probable sheet bounds, validates selected crop regions, classifies validation state as strong/weak/fail, applies perspective-aware crop extraction, and returns structured validation metadata to Flutter.
- **Thesis Objective Supported:** Image acquisition quality control and document preprocessing for lightweight OMR.
- **Key Algorithms/Logic:** OpenCV contour and page-bound detection; fallback bounds; crop geometry validation; music-sheet visual heuristics; brightness, horizontal-line, staff-like row, and edge-continuity checks.
- **Why It Is Important:** This file demonstrates the first research-critical stage: converting an uncontrolled camera/gallery image into a reliable sheet-music region. It also supports the user validation overlay behavior described in the system.
- **Relevance Score:** 10/10
- **Recommendation:** Full Appendix Inclusion

2. `android/app/src/main/kotlin/com/example/stala_app/StaffSegmentationProcessor.kt`

- **Purpose:** Extracts staff structures and supporting musical geometry from processed images.
- **Core Functionality:** Detects staff lines, validates five-line staff groups, identifies ledger lines, bar lines, stems, beams, measures, and produces overlay/debug data.
- **Thesis Objective Supported:** OMR structural analysis and music-region segmentation.
- **Key Algorithms/Logic:** Adaptive thresholding; horizontal and vertical morphology; row deduplication; staff-candidate construction; symbol-supported staff validation; ledger-line validation; stem/beam/barline detection; measure construction.
- **Why It Is Important:** This is one of the most technically significant files in the repository. It shows how STALA bridges raw image recognition and musical interpretation using custom structural analysis.

- **Relevance Score:** 10/10
- **Recommendation:** Full Appendix Inclusion

3. `android/app/src/main/kotlin/com/example/stala_app/OnnxDetector.kt`

- **Purpose:** Runs trained ONNX symbol detection on the cropped sheet image.
- **Core Functionality:** Loads the ONNX model, preprocesses the bitmap, applies CLAHE and letterboxing, creates the tensor input, executes inference, decodes detections, and returns bounding boxes/classes/confidence scores.
- **Thesis Objective Supported:** Lightweight music-symbol recognition.
- **Key Algorithms/Logic:** Image normalization; CLAHE enhancement; letterbox scaling; ONNX Runtime inference; bounding-box coordinate restoration; detection filtering.
- **Why It Is Important:** This file demonstrates the machine-learning recognition component that provides visual symbol candidates for the downstream music interpretation pipeline.
- **Relevance Score:** 9/10
- **Recommendation:** Full Appendix Inclusion

4. `lib/processing_page.dart`

- **Purpose:** Coordinates the complete recognition and translation pipeline from cropped image to generated tablature.
- **Core Functionality:** Calls native detection, invokes staff segmentation, filters symbol detections, runs translation services, builds debug snapshots, saves sessions, and navigates to debug or result pages.
- **Thesis Objective Supported:** End-to-end pipeline orchestration and application integration.
- **Key Algorithms/Logic:** Multi-stage workflow coordination; structural symbol filtering; notehead validation; clef-region safety filtering; ledger-note inference; processing status management; session snapshot construction.
- **Why It Is Important:** This is the central integration file. Although it contains UI code, its pipeline logic is crucial for explaining how the research modules work together.
- **Relevance Score:** 10/10
- **Recommendation:** Full Appendix Inclusion, or full inclusion with UI sections omitted if appendix length is constrained.

5. `lib/services/translation_grouping_service.dart`

- **Purpose:** Assigns detected symbols to staff, measure, segment, clef, and pitch context.
- **Core Functionality:** Groups detected music symbols by validated staff structure and prepares them for pitch, rhythm, and interpretation services.
- **Thesis Objective Supported:** Symbol-to-musical-context translation.
- **Key Algorithms/Logic:** Staff assignment; measure assignment; segment mapping; clef-aware grouping; symbol ordering; contextual metadata construction.
- **Why It Is Important:** This file shows how STALA converts geometric detections into musically meaningful grouped structures.
- **Relevance Score:** 9/10
- **Recommendation:** Full Appendix Inclusion

6. `lib/services/polyphonic_to_monophonic_service.dart`

- **Purpose:** Analyzes note groups and identifies harmonic stacks, melodic lines, and chord-aware structures.
- **Core Functionality:** Converts grouped note events into monophonic or chord-aware representations and detects chord/interval patterns.
- **Thesis Objective Supported:** Chord/melody handling and transformation of polyphonic notation into guitar-oriented events.
- **Key Algorithms/Logic:** Harmonic stack construction; pitch-class analysis; chord detection; dyad/interval classification; event simplification.
- **Why It Is Important:** This file directly supports the thesis claim that STALA handles pitch-based guitar tablature generation from notation that may contain simultaneous notes.
- **Relevance Score:** 9/10
- **Recommendation:** Full Appendix Inclusion

7. `lib/services/fretboard_mapping_service.dart`

- **Purpose:** Maps interpreted musical events to guitar string and fret candidates.
- **Core Functionality:** Converts pitch events into playable guitar positions and creates fretboard candidates for single notes and multi-note events.
- **Thesis Objective Supported:** Conversion from music interpretation to guitar tablature.
- **Key Algorithms/Logic:** Pitch-to-fretboard mapping; candidate generation; multi-pitch combinations; playable-position filtering.
- **Why It Is Important:** This is the core bridge from music recognition to guitar-specific output. It demonstrates the application-specific contribution beyond generic OMR.
- **Relevance Score:** 10/10
- **Recommendation:** Full Appendix Inclusion

8. `lib/services/chord_voicing_service.dart`

- **Purpose:** Selects playable chord voicings across events.
- **Core Functionality:** Scores fretboard candidates and chooses a low-cost path for chord positions across time.
- **Thesis Objective Supported:** Guitar playability and chord voicing optimization.
- **Key Algorithms/Logic:** Dynamic path selection; transition-cost scoring; fret-span penalties; skipped-string penalties; preference for practical chord shapes and root placement.
- **Why It Is Important:** This file contains one of the clearest custom algorithmic contributions in the guitar translation layer.
- **Relevance Score:** 10/10
- **Recommendation:** Full Appendix Inclusion

9. `lib/services/event_manager_service.dart`

- **Purpose:** Selects and organizes playable musical events from fretboard mapping results.
- **Core Functionality:** Converts mapped lines into managed event sequences and selects candidate positions for generated tablature output.
- **Thesis Objective Supported:** Playable event sequencing and tablature result preparation.
- **Key Algorithms/Logic:** Candidate selection; event ordering; per-line event construction; source-to-output linkage.
- **Why It Is Important:** This service is a compact but central step that converts candidate mappings into the playable event stream consumed by the tablature adapter.
- **Relevance Score:** 8/10
- **Recommendation:** Full Appendix Inclusion

10. `lib/services/generation_service.dart`

- **Purpose:** Formats tablature results into display-ready and export-ready structures.
- **Core Functionality:** Builds tablature rows, columns, event details, fretboard frames, measure separators, and export pages.
- **Thesis Objective Supported:** Final guitar tablature generation and visualization.
- **Key Algorithms/Logic:** Tab column layout; string-row generation; measure-start detection; fretboard highlight construction; export-page segmentation.
- **Why It Is Important:** This file demonstrates how interpreted and mapped events become a concrete tablature representation for users.
- **Relevance Score:** 9/10
- **Recommendation:** Full Appendix Inclusion

Recommended Files for Partial Snippet Inclusion

1. `lib/camera_logic.dart`

- **Purpose:** Implements camera/gallery capture and crop-preview workflow.
- **Core Functionality:** Requests permissions, captures images, imports gallery images, calls document detection/cropping methods, displays crop validation states.
- **Thesis Objective Supported:** User-facing image acquisition and validation workflow.
- **Key Algorithms/Logic:** Method-channel integration for `detectDocumentBounds`, `validateSelectedCrop`, and `cropDocumentImage`; strong/weak/fail validation handling; guarded long-press override.
- **Why It Is Important:** The full file includes substantial UI code, but selected snippets are important for documenting the acquisition-to-processing transition.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

2. `android/app/src/main/kotlin/com/example/stala_app/MainActivity.kt`

- **Purpose:** Bridges Flutter to native Android services through method channels.
- **Core Functionality:** Registers storage, accessibility, document detection, crop validation, crop generation, ONNX processing, and staff segmentation handlers.
- **Thesis Objective Supported:** Cross-layer application integration.
- **Key Algorithms/Logic:** Method-channel dispatch; background execution; Android storage access framework operations; native OMR method routing.
- **Why It Is Important:** Important architecturally, but too much of the file is platform plumbing. Include only channel registration and OMR handler snippets.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

3. `lib/services/rhythm_interpretation_service.dart`

- **Purpose:** Estimates rhythm events from grouped symbols and structural marks.
- **Core Functionality:** Uses note groups, stems, beams, and measure context to generate event timing data.
- **Thesis Objective Supported:** Musical interpretation and playback-ready tablature.
- **Key Algorithms/Logic:** Duration estimation; stem/beam-informed rhythm confidence; event labeling.
- **Why It Is Important:** This file supports the interpretation pipeline, especially where tablature playback duration is derived.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

4. `lib/services/musical_interpretation_service.dart`

- **Purpose:** Converts polyphonic/monophonic analysis into interpreted music lines.
- **Core Functionality:** Produces `Grand Staff` and `Treble Only` interpretation structures from chord-aware stacks.
- **Thesis Objective Supported:** Music interpretation layer between OMR data and guitar mapping.
- **Key Algorithms/Logic:** Interpretation line construction; fallback labels; mode separation.
- **Why It Is Important:** It is conceptually central but compact. Include the `interpret` method and result model rather than the full file if appendix space is limited.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

5. `lib/services/note_grouping_service.dart`

- **Purpose:** Groups symbols into note events.
- **Core Functionality:** Organizes translated note symbols into groups suitable for rhythm and interpretation.
- **Thesis Objective Supported:** Conversion from detected symbols to note structures.
- **Key Algorithms/Logic:** Staff-local note grouping; positional grouping; event-list construction.
- **Why It Is Important:** It is an intermediate algorithm supporting the larger interpretation pipeline.
- **Relevance Score:** 7/10
- **Recommendation:** Partial Snippet Inclusion

6. `lib/services/pitch_mapping_service.dart`

- **Purpose:** Maps staff positions to pitch labels.
- **Core Functionality:** Converts vertical staff/ledger placement into musical pitch names.
- **Thesis Objective Supported:** Pitch-based tablature generation.
- **Key Algorithms/Logic:** Staff-line position mapping; pitch-label lookup; ledger-aware pitch inference.
- **Why It Is Important:** It is small but directly tied to the thesis objective of pitch-based tablature.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

7. `lib/services/accidental_service.dart`

- **Purpose:** Resolves accidentals applied to note events.
- **Core Functionality:** Identifies sharp, flat, natural, or none from nearby symbols and applies them to pitch context.
- **Thesis Objective Supported:** Musical correctness in interpretation.
- **Key Algorithms/Logic:** Symbol proximity matching; accidental-state assignment.
- **Why It Is Important:** Include only if discussing pitch correctness beyond visual notehead mapping.
- **Relevance Score:** 7/10
- **Recommendation:** Partial Snippet Inclusion

8. `lib/services/key_signature_service.dart`

- **Purpose:** Interprets key signature candidates.
- **Core Functionality:** Identifies sharp/flat key signature patterns and returns key labels.
- **Thesis Objective Supported:** Music interpretation accuracy.
- **Key Algorithms/Logic:** Accidental count analysis; key-label classification.
- **Why It Is Important:** Useful as a supporting snippet in a music-interpretation appendix.
- **Relevance Score:** 6/10
- **Recommendation:** Partial Snippet Inclusion

9. `lib/services/grand_staff_pairing_service.dart`

- **Purpose:** Pairs treble and bass staffs into grand-staff units.
- **Core Functionality:** Associates related treble and bass staff groups for grand staff interpretation.
- **Thesis Objective Supported:** Grand staff-to-guitar translation.
- **Key Algorithms/Logic:** Staff-role pairing; treble/bass relationship construction.
- **Why It Is Important:** It directly supports the project's grand staff emphasis but is compact enough for snippet inclusion.
- **Relevance Score:** 7/10
- **Recommendation:** Partial Snippet Inclusion

10. `lib/services/barline_refinement_service.dart`

- **Purpose:** Refines detected barlines and measure boundaries.
- **Core Functionality:** Uses segmentation outputs and symbol context to improve barline/measure consistency.
- **Thesis Objective Supported:** Structural music segmentation.
- **Key Algorithms/Logic:** Barline filtering; measure-boundary refinement; duplicate handling.
- **Why It Is Important:** Strong supporting evidence for structural OMR robustness, but not as central as staff segmentation.
- **Relevance Score:** 7/10
- **Recommendation:** Partial Snippet Inclusion

11. `lib/services/tablature_result_adapter.dart`

- **Purpose:** Converts managed fretboard events and rhythm events into persistent tablature result objects.
- **Core Functionality:** Builds `TablatureResult`, `TablatureEvent`, and `TabPosition` records from interpreted/mapped event streams.
- **Thesis Objective Supported:** Final conversion of pipeline output into saved tablature data.
- **Key Algorithms/Logic:** Event-to-tab conversion; rhythm lookup; duration assignment; position transfer.
- **Why It Is Important:** It is the final adapter between algorithms and the saved result model.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

12. `lib/services/save_export_service.dart`

- **Purpose:** Saves STALA sessions and exports tablature as PNG, PDF, and ZIP packages.
- **Core Functionality:** Serializes sessions, renders tablature images, builds PDFs, packages ZIP archives, and writes through storage access.
- **Thesis Objective Supported:** Output preservation and reproducible user results.
- **Key Algorithms/Logic:** Export page rendering; PDF construction; ZIP manifest creation; file-safe naming.
- **Why It Is Important:** Include snippets for `.stala` serialization and PNG/PDF rendering. The full file is lengthy and includes export drawing details that can be summarized.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

13. `lib/models/session_data.dart`

- **Purpose:** Defines the persistent session object.
- **Core Functionality:** Stores source image paths, pipeline snapshots, detected symbols, segmentation data, fretboard events, tablature results, timestamps, model version, and auto-save metadata.
- **Thesis Objective Supported:** Traceability and reproducibility of OMR outputs.
- **Key Algorithms/Logic:** Structured serialization/deserialization; immutable copy pattern.
- **Why It Is Important:** It documents what the system considers a complete translation session.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

14. `lib/models/tablature_result.dart`

- **Purpose:** Defines the persistent tablature result data model.
- **Core Functionality:** Represents translation modes, events, durations, guitar positions, and chord/rest state.
- **Thesis Objective Supported:** Formal output model for pitch-based guitar tablature.
- **Key Algorithms/Logic:** Translation-mode mapping; result/event/position serialization.
- **Why It Is Important:** Include alongside generation or adapter snippets to define the output structure.
- **Relevance Score:** 8/10
- **Recommendation:** Partial Snippet Inclusion

15. `lib/data/recent_items_repository.dart`

- **Purpose:** Manages saved STALA sessions, import, rename, delete, duplicate detection, and bulk ZIP import/export support.
- **Core Functionality:** Loads `.stala` sessions, imports `.stala` and ZIP archives, validates duplicate filenames, and updates stored project titles.
- **Thesis Objective Supported:** Session management and practical deployment workflow.
- **Key Algorithms/Logic:** Safe filename generation; duplicate-title checks; archive parsing; session loading from JSON.
- **Why It Is Important:** Relevant for demonstrating user workflow persistence, but less central than OMR and interpretation algorithms.
- **Relevance Score:** 7/10
- **Recommendation:** Partial Snippet Inclusion

Files Recommended for Architectural Discussion Only

1. `lib/result_page.dart`

- **Purpose:** Presents generated tablature, playback controls, fretboard map, title editing, and export buttons.
- **Core Functionality:** Result visualization and user interaction.
- **Thesis Objective Supported:** Result review and application integration.
- **Key Algorithms/Logic:** Playback progression, fretboard hit testing, tab interaction, export invocation.
- **Why It Is Important:** It validates that generated outputs are usable, but most of the file is UI code.
- **Relevance Score:** 6/10
- **Recommendation:** Architectural Discussion Only, with at most a small snippet for playback/fretboard interaction.

2. `lib/dummy_page.dart`

- **Purpose:** Provides debug visualization for pipeline outputs.
- **Core Functionality:** Displays input crop, detection overlays, staff validation, structural segmentation, musical interpretation, tablature generation, and reports.
- **Thesis Objective Supported:** Pipeline validation and diagnostic transparency.
- **Key Algorithms/Logic:** Overlay painters and debug-panel organization.
- **Why It Is Important:** Strong for figures/screenshots and system validation discussion, but too UI-heavy for appendix source inclusion.
- **Relevance Score:** 6/10
- **Recommendation:** Architectural Discussion Only

3. `lib/services/staff_segmentation_service.dart`

- **Purpose:** Dart bridge to native staff segmentation with fallback behavior.
- **Core Functionality:** Calls the native segmentation method and normalizes results.
- **Thesis Objective Supported:** Native-Dart integration for staff segmentation.
- **Key Algorithms/Logic:** Method-channel invocation and fallback segmentation.
- **Why It Is Important:** Discuss as part of architecture, but the primary algorithm is in `StaffSegmentationProcessor.kt`.
- **Relevance Score:** 6/10

- **Recommendation:** Architectural Discussion Only

4. `lib/services/storage_access_service.dart`

- **Purpose:** Provides Dart API for Android storage access methods.
- **Core Functionality:** Selects folders, imports documents, reads/writes/deletes/renames files, and lists storage documents.
- **Thesis Objective Supported:** Integration and export workflow.
- **Key Algorithms/Logic:** Method-channel wrappers and typed result models.
- **Why It Is Important:** Architecturally relevant but mostly platform abstraction.
- **Relevance Score:** 5/10
- **Recommendation:** Architectural Discussion Only

5. `lib/menu_page.dart`

- **Purpose:** Main dashboard for Home, Import, Settings, and storage/permission controls.
- **Core Functionality:** Recent sessions, import list, settings panels, and storage-folder prompts.
- **Thesis Objective Supported:** Complete user workflow and application navigation.
- **Key Algorithms/Logic:** Workflow routing and repository/service integration.
- **Why It Is Important:** Useful to discuss application structure, but it is primarily UI and state management.
- **Relevance Score:** 5/10
- **Recommendation:** Architectural Discussion Only

6. `android/app/src/main/kotlin/com/example/stala_app/PipelineProcessor.kt`

- **Purpose:** Provides an older or simplified native image processing response wrapper.
- **Core Functionality:** Produces standardized success/error maps.
- **Thesis Objective Supported:** Native processing response structure.
- **Key Algorithms/Logic:** Error response format and basic image validation.
- **Why It Is Important:** Useful background only; the primary OMR implementation is represented better by `OnnxDetector.kt` and `StaffSegmentationProcessor.kt`.
- **Relevance Score:** 4/10
- **Recommendation:** Architectural Discussion Only

7. `android/app/src/main/kotlin/com/example/stala_app/ImageDecodeUtils.kt`

- **Purpose:** Decodes images with orientation correction.
- **Core Functionality:** Handles bitmap decoding and EXIF orientation normalization.
- **Thesis Objective Supported:** Image preprocessing reliability.
- **Key Algorithms/Logic:** Orientation-aware bitmap loading.
- **Why It Is Important:** Supporting utility; include only in architecture discussion if explaining preprocessing robustness.
- **Relevance Score:** 4/10
- **Recommendation:** Architectural Discussion Only

Files That Should NOT Be Included

The following file groups should be excluded from thesis appendices unless they are needed for a brief architecture diagram or environment note:

- `opencv/` vendor tree: external library code, not STALA contribution.
- `build/`, `.dart_tool/`, `.idea/`, `.git/`: generated or environment-specific files.
- `android/app/src/main/java/io/flutter/plugins/GeneratedPluginRegistrant.java`: generated Flutter plugin registration.
- Launcher icons and splash assets in `android/app/src/main/res/mipmap-*` and `assets/images/`: visual assets, not algorithmic source.
- Theme-only files such as `lib/core/theme/app_colors.dart` and `lib/core/theme/app_text_styles.dart`: useful for UI consistency but not thesis-relevant code.
- `lib/main.dart`, `lib/pages/splash_page.dart`, `lib/camera_panel.dart`, `lib/app_restart_widget.dart`: app shell or wrappers with minimal thesis value.
- `lib/services/tutorial_service.dart`: user guidance system; relevant to usability but not the core OMR/tabs contribution.
- `lib/services/audio_playback_service.dart`: playback support; useful feature but not central to recognition or tablature generation.
- `lib/data/app_settings_repository.dart` and `lib/data/debug_settings_repository.dart`: preference storage only.
- `lib/models/saved_item_data.dart`: simple saved-item display model.
- `test/widget_test.dart`: default or lightweight test scaffold, not representative of the research system.
- `README.md`, `pubspec.yaml`, Gradle files, manifests, and XML resources: configuration and metadata.
- `android/app/src/main/kotlin/com/example/stala_app/MyAccessibilityService.kt`: accessibility service stub, not central to thesis objectives.

TOP 10 MOST IMPORTANT FILES

1. `android/app/src/main/kotlin/com/example/stala_app/StaffSegmentationProcessor.kt`
2. `android/app/src/main/kotlin/com/example/stala_app/DocumentProcessor.kt`
3. `lib/processing_page.dart`
4. `lib/services/fretboard_mapping_service.dart`
5. `lib/services/chord_voicing_service.dart`
6. `android/app/src/main/kotlin/com/example/stala_app/OnnxDetector.kt`
7. `lib/services/translation_grouping_service.dart`
8. `lib/services/polyphonic_to_monophonic_service.dart`
9. `lib/services/generation_service.dart`
10. `lib/services/event_manager_service.dart`

TOP 5 MOST IMPORTANT CODE SNIPPETS

1. **Staff segmentation and structural feature extraction**

   - Source: `android/app/src/main/kotlin/com/example/stala_app/StaffSegmentationProcessor.kt`

   - Suggested snippet: `segmentStaffLines`, `buildValidatedStaffs`, `detectLedgerLines`, `detectStems`, `detectBeams`, and `buildMeasures`.

   - Reason: Best demonstrates custom OMR structural analysis.

2. **Document detection and crop validation**

   - Source: `android/app/src/main/kotlin/com/example/stala_app/DocumentProcessor.kt`

   - Suggested snippet: `detectDocumentBounds`, `validateSelectedCrop`, `validateBoundsOnBitmap`, and `cropDocumentImage`.

   - Reason: Shows image-preprocessing safeguards and music-sheet-aware validation.

3. **End-to-end processing orchestration**

   - Source: `lib/processing_page.dart`

   - Suggested snippet: `_startProcessingPipeline`, `_filterStructureAwareDetections`, `_validateNoteheadCandidate`, and `_generateInferredLedgerNoteheads`.

   - Reason: Shows how native recognition, staff segmentation, symbolic validation, interpretation, mapping, and saving are connected.

4. **Fretboard mapping and chord voicing**

   - Sources: `lib/services/fretboard_mapping_service.dart` and `lib/services/chord_voicing_service.dart`

   - Suggested snippet: `mapInterpretation`, `_multiPitchCandidates`, `voice`, `_findLowestCostPath`, and `_scoreCandidate`.

   - Reason: Demonstrates the guitar-specific contribution and playability optimization.

5. **Tablature generation and export model**

   - Sources: `lib/services/generation_service.dart`, `lib/services/tablature_result_adapter.dart`, and `lib/models/tablature_result.dart`

   - Suggested snippet: `generate`, `generateAll`, `_buildExportPages`, `_fromPlayableEvent`, and `TablatureResult` / `TablatureEvent` model definitions.

   - Reason: Shows how interpreted events become displayable and persistent tablature.

Exact Source Code Syntax Snippets

The following excerpts are exact source code snippets from the repository. They are intended for thesis appendix use as representative syntax examples, not as a replacement for full-file appendix inclusion where full inclusion is recommended. Some excerpts stop before the full method ends because the complete method is lengthy; in those cases, the displayed lines are still copied directly from the source file.

Snippet 1 - Document Detection Entry Point

Source File: `android/app/src/main/kotlin/com/example/stala_app/DocumentProcessor.kt`

```kotlin
fun detectDocumentBounds(imagePath: String): Map<String, Any?> {
    Log.d("DocumentProcessor", "=== detectDocumentBounds called ===")
    Log.d("DocumentProcessor", "imagePath=$imagePath")

    val imageFile = File(imagePath)
    if (!imageFile.exists()) {
        return detectionFailure("Image file does not exist.")
    }

    val bitmap = ImageDecodeUtils.decodeBitmapWithCorrectOrientation(imagePath)
        ?: return detectionFailure("Failed to decode image.")

    if (bitmap.width < 200 || bitmap.height < 200) {
        bitmap.recycle()
        return detectionFailure("Image is too small for document detection.")
    }

    val targetWidth = 400
    val scale = targetWidth.toFloat() / bitmap.width.toFloat()
    val scaledHeight = (bitmap.height * scale).toInt().coerceAtLeast(1)
    val scaledBitmap = bitmap.scale(targetWidth, scaledHeight)
    bitmap.recycle()

    Log.d(
        "DocumentProcessor",
        "scaledWidth=${scaledBitmap.width} scaledHeight=${scaledBitmap.height}"
    )

    val openCvBounds = findOpenCvDocumentBounds(scaledBitmap)
    val validOpenCvBounds = openCvBounds?.takeIf {
        isAcceptableOpenCvBounds(it, scaledBitmap)
    }
```

Snippet 2 - Second-Pass Crop Validation

Source File: `android/app/src/main/kotlin/com/example/stala_app/DocumentProcessor.kt`

```
fun validateSelectedCrop(
    imagePath: String,
    bounds: Map<String, Any?>
): Map<String, Any?> {

    Log.d("DocumentProcessor", "=== validateSelectedCrop called ===")

    val imageFile = File(imagePath)
    if (!imageFile.exists()) {
        return validationFailure("Image file does not exist.")
    }

    val bitmap = ImageDecodeUtils.decodeBitmapWithCorrectOrientation(imagePath)
        ?: return validationFailure("Failed to decode image.")

    try {
        val rect = boundsToRect(bitmap, bounds) ?: return validationFailure(
            "The selected crop is not yet a reliable music-sheet region. Please
adjust the box."
        )

        val left = rect[0]
        val top = rect[1]
        val right = rect[2]
        val bottom = rect[3]

        val cropWidth = (right - left).coerceAtLeast(1)
        val cropHeight = (bottom - top).coerceAtLeast(1)

        val marginX = (cropWidth * 0.02).toInt()
        val marginY = (cropHeight * 0.02).toInt()
```

Snippet 3 - Staff Segmentation Entry Point

Source File:

`android/app/src/main/kotlin/com/example/stala_app/StaffSegmentationProcessor.kt`

```
fun segmentStaffLines(
    context: Context,
    imagePath: String,
    symbolDetections: List<Map<String, Any?>> = emptyList()
): Map<String, Any?> {

    val src = Imgcodecs.imread(imagePath)
    if (src.empty()) {
        return error("Failed to load image")
    }

    val gray = Mat()
    Imgproc.cvtColor(src, gray, Imgproc.COLOR_BGR2GRAY)

    val blurred = Mat()
    Imgproc.GaussianBlur(gray, blurred, Size(3.0, 3.0), 0.0)

    val binary = Mat()
    Imgproc.threshold(
        blurred,
        binary,
        0.0,
        255.0,
        Imgproc.THRESH_BINARY_INV or Imgproc.THRESH_OTSU
    )

    val horizontal = binary.clone()
    val kernelWidth = (src.cols() / 12).coerceAtLeast(25)
    val horizontalKernel = Imgproc.getStructuringElement(
        Imgproc.MORPH_RECT,
        Size(kernelWidth.toDouble(), 1.0)
    )

    Imgproc.morphologyEx(horizontal, horizontal, Imgproc.MORPH_OPEN,
horizontalKernel)
```

Snippet 4 - Processing Pipeline Orchestration

Source File: `lib/processing_page.dart`

```dart
Future<void> _startProcessingPipeline() async {
  if (_isProcessingActive) return;
  _isProcessingActive = true;
  try {
    if (!mounted) return;

    setState(() {
      _activeStageIndex = 0;
      _statusMessage = 'Getting your music sheet ready...';
      _stages[0] = _stages[0].copyWith(status: ProcessingStageStatus.active);
    });

    await Future.delayed(const Duration(milliseconds: 400));

    if (!mounted) return;

    setState(() {
      _stages[0] = _stages[0].copyWith(
        status: ProcessingStageStatus.completed,
      );
      _activeStageIndex = 1;
      _statusMessage = 'Finding the notes and symbols on the page...';
      _stages[1] = _stages[1].copyWith(status: ProcessingStageStatus.active);
    });

    setState(() {
      _statusMessage = 'Scanning the music symbols...';
    });

    final dynamic result = await _visionPipelineChannel.invokeMethod(
      'processImage',
      {'imagePath': widget.croppedImagePath},
    );
```

Snippet 5 - Structure-Aware Symbol Filtering

Source File: `lib/processing_page.dart`

```dart
_DetectionFilterResult _filterStructureAwareDetections({
  required List<dynamic> rawDetections,
  required List<dynamic> validatedStaffs,
  required List<dynamic> ledgerLines,
  required List<dynamic> stems,
  required List<dynamic> beams,
}) {
  final symbols = rawDetections
      .whereType<Map>()
      .map((item) {
        return Map<String, dynamic>.from(
          item.map((key, value) => MapEntry(key.toString(), value)),
        );
      })
      .where((item) => _symbolGeometry(item) != null)
      .toList();

  print('STALA_COUNTS: AFTER STAFF ASSOCIATION symbols=${symbols.length}');

  final clefs = symbols.where((item) {
    final className = _symbolClassName(item);
    return className == 'treble_clef' || className == 'bass_clef';
  }).toList();

  final semanticRegions = _buildPreMeasureSemanticRegions(
    clefs: clefs,
    validatedStaffs: validatedStaffs,
  );
  final clefSafetyRegions = _buildClefSafetyRegions(
    clefs: clefs,
    validatedStaffs: validatedStaffs,
  );
```

Snippet 6 - Fretboard Mapping

Source File: `lib/services/fretboard_mapping_service.dart`

```
FretboardMappingResult mapInterpretation({
  required MusicalInterpretationResult interpretation,
}) {
  final lines = [
    _mapLine(interpretation.grandStaffLine),
    _mapLine(interpretation.trebleOnlyLine),
  ];

  return FretboardMappingResult(lines: lines);
}

FretboardMappedLine _mapLine(InterpretedMusicLine line) {
  return FretboardMappedLine(
    id: line.id,
    title: line.title,
    events: line.events.map(_mapEvent).toList(),
  );
}

FretboardMappedEvent _mapEvent(InterpretedMusicEvent event) {
  final candidates = <FretboardCandidate>[];

  if (event.pitches.length == 1) {
    final pitch = event.pitches.first;
    final positions = _positionsForPitch(pitch);

    for (final pos in positions) {
      candidates.add(
        FretboardCandidate(
          label: '${pos.pitch}: S${pos.stringNumber} F${pos.fret}',
          positions: [pos],
        ),
      );
    }
  } else {
    candidates.addAll(_multiPitchCandidates(event));
  }
```

Snippet 7 - Chord Voicing Dynamic Path Selection

Source File: `lib/services/chord_voicing_service.dart`

```dart
ChordVoicingResult voice({required FretboardMappingResult fretboardMapping}) {
  final lines = fretboardMapping.lines
      .where((line) => line.id.contains('chord'))
      .map(_voiceLine)
      .whereType<ChordVoicingLine>()
      .toList();

  return ChordVoicingResult(lines: lines);
}

List<FretboardCandidate> _findLowestCostPath(
  List<FretboardMappedEvent> events,
) {
  final dp = <Map<int, _PathState>>[];

  final firstStates = <int, _PathState>{};
  for (int i = 0; i < events.first.candidates.length; i++) {
    final candidate = events.first.candidates[i];
    firstStates[i] = _PathState(
      cost: _scoreCandidate(candidate).cost,
      previousIndex: null,
    );
  }
  dp.add(firstStates);

  for (int eventIndex = 1; eventIndex < events.length; eventIndex++) {
    final previousCandidates = events[eventIndex - 1].candidates;
    final currentCandidates = events[eventIndex].candidates;
    final currentStates = <int, _PathState>{};
```

Snippet 8 - Tablature Generation

Source File: `lib/services/generation_service.dart`

```dart
GeneratedTabResult generate({
  required TablatureResult result,
  double columnWidth = 48,
  double rowHeight = 32,
  int exportEventsPerPage = 24,
}) {
  final columns = <GeneratedTabColumn>[];

  for (int i = 0; i < result.events.length; i++) {
    final event = result.events[i];
    final previous = i > 0 ? result.events[i - 1] : null;
    final startsMeasure = _startsMeasure(event, previous);
    final measureGap = startsMeasure && i > 0 ? columnWidth * 0.45 : 0.0;
    final eventWidth = _widthForDuration(event.durationSeconds, columnWidth);
    final x =
        (columns.isEmpty ? 0.0 : columns.last.x + columns.last.width) +
        measureGap;

    columns.add(
      GeneratedTabColumn(
        eventIndex: event.eventIndex,
        label: event.label,
        measureIndex: _metadataInt(event, 'measureIndex'),
        startsMeasure: startsMeasure,
        durationSeconds: event.durationSeconds,
        x: x,
        width: eventWidth,
        numbers: _buildNumbers(event: event, x: x + (eventWidth / 2)),
        eventDetail: EventDetail(
          eventIndex: event.eventIndex,
          label: event.label,
          durationSeconds: event.durationSeconds,
          positions: event.positions,
        ),
      ),
    );
  }
```

Suggested Appendix Structure

- **Appendix H - Detection Pipeline**
  - `DocumentProcessor.kt`
  - `OnnxDetector.kt`
  - Selected `MainActivity.kt` method-channel snippets
- **Appendix I - Staff Segmentation and Structural Validation**
  - `StaffSegmentationProcessor.kt`
  - Selected `processing_page.dart` snippets for symbol validation and ledger-note inference
  - Optional debug screenshots from `dummy_page.dart`
- **Appendix J - Music Interpretation Logic**
  - `translation_grouping_service.dart`
  - `note_grouping_service.dart` snippets
  - `rhythm_interpretation_service.dart` snippets
  - `polyphonic_to_monophonic_service.dart`
  - `musical_interpretation_service.dart` snippets
  - `pitch_mapping_service.dart`, `accidental_service.dart`, `key_signature_service.dart`, and `grand_staff_pairing_service.dart` snippets as supporting material
- **Appendix K - Guitar Tablature Generation**
  - `fretboard_mapping_service.dart`
  - `chord_voicing_service.dart`
  - `event_manager_service.dart`
  - `tablature_result_adapter.dart` snippets
  - `generation_service.dart`
  - `tablature_result.dart` model snippets
- **Appendix L - Export and Session Management**
  - `session_data.dart` snippets
  - `save_export_service.dart` snippets for `.stala`, PNG, PDF, and ZIP export
  - `recent_items_repository.dart` snippets for import, rename, duplicate checking, and session loading

Recommended Appendix Strategy

The thesis appendices should not include the entire codebase. The strongest strategy is to include a small number of full, algorithm-heavy files and use snippets for supporting services. Full inclusion should be reserved for source files that clearly show original technical work: document processing, staff segmentation, symbol detection, processing orchestration, fretboard mapping, chord voicing, and tablature generation.

UI-heavy files should be discussed architecturally and supported by screenshots rather than printed in full. This keeps the appendices focused on research contribution, implementation rigor, and traceable system behavior while avoiding unnecessary boilerplate.